

Documentation du Projet de Gestion de Contacts

Zbida Mohamed Amine - Rjili Houssam

5 janvier 2025

Introduction

Le projet de gestion de contacts permet aux utilisateurs de gérer un carnet d'adresses électronique. Il inclut des fonctionnalités telles que l'importation, la suppression et l'affichage de contacts, ainsi que la gestion dynamique des catégories associées à chaque contact. Ce projet propose une interface intuitive qui facilite l'interaction avec les données de contacts.

1 Description du Projet

Le projet a pour objectif de faciliter la gestion des contacts à travers une interface web. Les utilisateurs peuvent importer des contacts depuis des fichiers JSON ou CSV, attribuer des catégories à chaque contact, et supprimer des contacts de manière interactive.

1.1 Fonctionnalités Principales

- Importation de contacts à partir de fichiers JSON et CSV.
- Attribution et gestion de catégories pour chaque contact.
- Suppression de contacts, soit individuellement, soit en groupe.
- Filtrage dynamique des contacts par catégorie.
- Interface utilisateur réactive avec des alertes et animations pour les actions de l'utilisateur.

2 Architecture du Système

2.1 Vue d'ensemble

Le projet adopte une architecture côté client-serveur. L'application fonctionne sur un modèle de *client léger* où l'interface utilisateur est responsable de l'interaction avec l'utilisateur, tandis que le backend gère la logique des données et la communication avec la base de données.

2.2 Composants de l'Architecture

- **Frontend** : L'interface utilisateur est construite en HTML, CSS et JavaScript. Elle permet de visualiser et d'interagir avec les contacts. Les actions de l'utilisateur sont traitées par JavaScript, qui communique avec le backend via des requêtes HTTP.
- **Backend** : Le serveur est implémenté en PHP et interagit avec une base de données pour stocker les contacts. Le backend reçoit des données au format JSON et exécute des actions comme l'ajout, la suppression ou la mise à jour des contacts.
- **Base de données** (optionnel) : Une base de données relationnelle (MySQL ou SQLite) est utilisée pour stocker les contacts de manière persistante.

2.3 Flux de Travail

Le processus de gestion des contacts se déroule en plusieurs étapes clés :

1. L'utilisateur sélectionne un fichier d'importation (JSON ou CSV).
2. Les données du fichier sont lues et traitées par JavaScript.
3. Les contacts sont envoyés au serveur via une requête HTTP POST.
4. Le serveur ajoute ou supprime les contacts et renvoie une réponse au frontend.
5. Le frontend met à jour l'affichage en fonction des résultats reçus.

3 Technologies Utilisées

3.1 Frontend

- **HTML/CSS** : Structure et présentation de l'interface utilisateur.
- **JavaScript (Vanilla)** : Gestion des interactions avec l'utilisateur (importation de fichiers, suppression de contacts, etc.).
- **Fetch API** : Permet d'envoyer des requêtes HTTP vers le serveur et de recevoir des réponses.
- **CSS Transitions** : Pour ajouter des animations lors de la suppression des contacts.

3.2 Backend

- **PHP** : Le backend utilise PHP pour traiter les requêtes et gérer les contacts. Il interagit avec la base de données pour insérer, supprimer ou modifier les contacts.
- **MySQL/SQLite** (optionnel) : Utilisation d'une base de données relationnelle pour la gestion des contacts de manière persistante.
- **JSON** : Format d'échange de données entre le frontend et le backend.

3.3 Outils Supplémentaires

- **Set et Map (JavaScript)** : Structures de données pour gérer efficacement les catégories et les contacts sélectionnés.

4 Base de données Utilisées

4.1 MCD & MLD

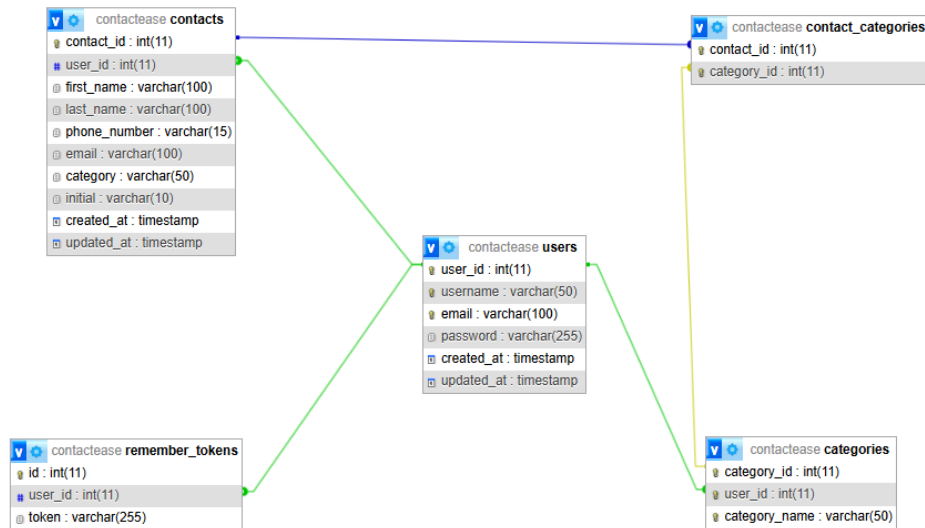


FIGURE 1 – Model Logique de données

Table : contacts

- **Relation avec users :**
 - Un utilisateur (**users**) peut avoir plusieurs contacts (**contacts**).
 - **Cardinalité** : 1 : N (un utilisateur - plusieurs contacts).
- **Relation avec contact_categories :**
 - Un contact peut appartenir à plusieurs catégories via la table intermédiaire **contact_categories**.
 - **Cardinalité** : N : N (plusieurs contacts - plusieurs catégories).

Table : `contact_categories` (table d'association)

- Relie les tables `contacts` et `categories`.
- Chaque enregistrement associe un contact (`contact_id`) à une catégorie (`category_id`).
- **Cardinalité** : $N : N$.

Table : `categories`

- **Relation avec `users`** :
 - Un utilisateur peut créer plusieurs catégories.
 - **Cardinalité** : $1 : N$ (un utilisateur - plusieurs catégories).
- **Relation avec `contact_categories`** :
 - Une catégorie peut être associée à plusieurs contacts via `contact_categories`.
 - **Cardinalité** : $N : N$ (plusieurs contacts - plusieurs catégories).

Table : `remember_tokens`

- **Relation avec `users`** :
 - Un utilisateur peut avoir plusieurs tokens.
 - **Cardinalité** : $1 : N$ (un utilisateur - plusieurs tokens).

Synthèse des cardinalités

- `users` ↔ `contacts` : $1 : N$.
- `users` ↔ `categories` : $1 : N$.
- `contacts` ↔ `categories` : $N : N$ (via `contact_categories`).
- `users` ↔ `import_export_history` : $1 : N$.
- `users` ↔ `remember_tokens` : $1 : N$.

5 Détails de l'Implémentation en Bref

5.1 Importation des Contacts

L'importation de contacts est effectuée en lisant un fichier au format JSON ou CSV sélectionné par l'utilisateur. Une fois les données extraites, elles sont envoyées au backend via une requête HTTP POST en format JSON. Le backend les enregistre dans la base de données ou dans une structure de données temporaire.

5.2 Suppression des Contacts

Les contacts peuvent être supprimés individuellement ou en masse. Lorsqu'un utilisateur confirme la suppression d'un contact, une requête est envoyée au serveur pour supprimer le contact de la base de données. Simultanément, une animation visuelle est effectuée sur l'élément du contact supprimé pour améliorer l'expérience utilisateur.

5.3 Gestion Dynamique des Catégories

Les catégories des contacts sont récupérées dynamiquement depuis le backend. Si une nouvelle catégorie est ajoutée, elle est automatiquement ajoutée au menu déroulant de sélection des catégories. Cela permet à l'utilisateur de gérer ses contacts avec une flexibilité maximale.

6 Conclusion

Le projet de gestion de contacts offre une solution complète et interactive pour gérer des contacts électroniques. Grâce à l'utilisation des technologies web modernes telles que HTML, CSS, JavaScript, et PHP, il permet une gestion dynamique et fluide des contacts. L'architecture modulaire et l'interface réactive assurent une expérience utilisateur agréable et efficace.